E.E.S.T 1 | 5°3° 2026

Actividad N7: Manual de Usuario

Martina Araujo, Thiago Goya y Sofia Salaberry,

Participantes:
Martina Araujo, Thiago Goya y Sofia Salaberry

Escuela:
Escuela Técnica N°1 de Vicente Lopez

Curso:
Ciclo Superior 5°3°

Docentes a cargo:
Yamil Ganduglia, York Elias Mansilla Muñoz

Materia:
Lab. Programación

Año: 2026

Repositorio de GitHub:
Goya: https://github.com/GoyaYuki/Mini_juego

E.E.S.T 1 | 5°3° 2026

Actividad N7: Manual de Usuario

Martina Araujo, Thiago Goya y Sofia Salaberry,

Objetivo del manual

El objetivo de este manual es explicar cómo está desarrollado el programa de la Actividad 7 y cómo se utilizan las estructuras (struct), las funciones y los archivos en C++.

El programa permite jugar Piedra, Papel o Tijera contra la inteligencia artificial de la consola. Cada partida se organiza mediante una estructura llamada Partida y se guarda en historial_partidas.txt para poder consultarla más adelante.

Estructura utilizada en el programa

```
struct Partida {
    int idPartida;
    string jugador;
    string eleccionJugador;
    string eleccionIA;
    string resultado;
};
```

Los elementos que forman la estructura son:

- **idPartida**: número que identifica cada partida.
- **jugador**: nombre o nickname de la persona que juega.
- **eleccionJugador**: jugada elegida por el usuario.
- **eleccionIA**: jugada elegida aleatoriamente por la computadora.
- **resultado**: indica si terminó en Victoria, Derrota o Empate.

Creación de la partida

En jugarPartida() se crea una variable de tipo Partida:

```
Partida p;
```

Después, se asigna un identificador:

```
p.idPartida = contadorID++;
```

El contador comienza en 1 y aumenta cada vez que se inicia una partida durante la ejecución. Al cerrar y volver a abrir el programa, vuelve a comenzar en 1 porque es una variable temporal y no se recupera desde el archivo.

Organización del programa mediante funciones

El programa se divide en funciones para ordenar las tareas principales:

- main(): muestra el menú y permite elegir una opción.
- jugarPartida(int& contadorID): solicita el nombre, recibe la jugada del usuario, genera la jugada de la IA, calcula el resultado y manda a guardar la partida.
- guardarEnDisco(const Partida& p): escribe los datos en el archivo de texto.
- leerHistorialDisco(): lee el archivo y muestra las partidas guardadas.

Las funciones separan las responsabilidades y facilitan la lectura y el mantenimiento del código.

E.E.S.T 1 | 5°3° 2026

Actividad N7: Manual de Usuario

Martina Araujo, Thiago Goya y Sofia Salaberry,

Funcionamiento de main()

En main() se inicializa la generación de números aleatorios con srand(time(0)). Luego se declaran contadorID y opción y se muestra el menú principal.

Las opciones son:

1. Jugar una partida;
2. Ver el historial guardado en disco;
3. Salir del programa.

El menú se repite mediante un ciclo do-while mientras la opción elegida sea distinta de 3.

Desarrollo de una partida

Cuando se elige la opción 1, se ejecuta jugarPartida(). La función pide un nombre o nickname y presenta las opciones Piedra, Papel y Tijera.

El nombre se lee con getline(cin, p.jugador), que permite ingresar nombres con espacios. Antes se utiliza cin.ignore() para descartar el salto de línea pendiente después de leer la opción del menú. Para elegir la jugada de la IA se utiliza:

```
int opcIA = (rand() % 3) + 1;
```

Esto genera un número aleatorio entre 1 y 3, que luego se convierte en Piedra, Papel o Tijera.

Determinación del resultado

El programa compara la elección del jugador con la de la IA. Si son iguales, el resultado es empate.

Si el jugador elige Piedra contra Tijera, Papel contra Piedra o Tijera contra Papel, el resultado es Victoria. En los demás casos, el resultado es Derrota.

Al terminar, se muestra el resultado en la consola y se llama a guardarEnDisco(p) para registrar la partida.

Persistencia de datos: RAM y disco

Las variables del programa se almacenan temporalmente en la memoria RAM. Cuando se cierra el programa, esos valores dejan de estar disponibles.

Para conservar las partidas, el programa escribe los datos en historial_partidas.txt. El archivo está guardado en el disco, por lo que su contenido permanece aunque se cierre la consola o se vuelva a ejecutar el programa.

El código de referencia utiliza la ruta relativa src/historial_partidas.txt. La carpeta src debe existir y el programa debe ejecutarse desde una ubicación compatible con esa ruta.

Escritura en disco: ofstream e ios::app

```
ofstream archivo("../src/historial_partidas.txt", ios::app);
```

ofstream es un flujo de salida que permite escribir en un archivo. ios::app indica que los nuevos datos se agregan al final sin borrar las partidas anteriores.

Actividad N7: Manual de Usuario

Martina Araujo, Thiago Goya y Sofia Salaberry,

Los campos se guardan separados por comas:

```
archivo << p.idPartida << ","
        << p.jugador << ","
        << p.eleccionJugador << ","
        << p.eleccionIA << ","
        << p.resultado << endl;
```

Después de escribir, se cierra el archivo con archivo.close(). Si no se puede abrir, se muestra un mensaje de error y la función termina.

Lectura del historial: ifstream

```
ifstream archivo("../src/historial_partidas.txt");
```

ifstream permite recuperar información guardada en un archivo. Si no se puede abrir, el programa avisa que todavía no existen partidas guardadas.

Si se abre correctamente, getline() utiliza las comas como separadores para recuperar los campos. Luego muestra el número de partida, el jugador, las elecciones y el resultado. Al final informa cuántos registros leyó y cierra el archivo.

Buffer de entrada: cin.ignore() y getline()

Después de cin >> opcion queda un salto de línea (Enter) en el búfer. Si se llama directamente a getline(), ese salto puede ser leído y la pregunta del nombre parece saltearse.

Por eso se usa cin.ignore() antes de getline(cin, p.jugador). Así se descarta el carácter pendiente y se puede ingresar correctamente el nombre completo.

Errores comunes

1. No se crea el archivo: se genera al guardar la primera partida. Revisar que la carpeta src exista y que la ruta sea correcta.
2. El historial no conserva las partidas: comprobar que guardarEnDisco() abra el archivo con ios::app.
3. Se saltea el nombre: colocar cin.ignore() antes de getline().
4. No se puede abrir el archivo: verificar la ruta y los permisos de escritura.
5. El menú se repite o no acepta una opción: si se ingresa texto donde se espera un número, cin puede quedar en estado de error. El código de referencia no incluye una validación completa de ese caso.

Conclusión

El programa aplica los conceptos de struct, funciones, generación de números aleatorios y manejo de archivos con fstream.

La estructura Partida organiza los datos de cada jugada, ofstream junto con ios::app permite guardar nuevas partidas sin borrar el historial e ifstream permite recuperar los registros en ejecuciones posteriores. Así se diferencia la información temporal de la RAM de los datos persistentes guardados en disco.

E.E.S.T 1 | 5°3° 2026

Actividad N7: Manual de Usuario

Martina Araujo, Thiago Goya y Sofia Salaberry,